

HW/SW Co-Design Project

Benchmark report: nbody

pyperformance optimization and hardware acceleration (V3A)

Software speedup	Run time	Correctness	Hardware (V3A)
1.64x	229 ms -> 140 ms	energy diff 2.2e-16	38 cycles/step, ~18x

Environment: Linux-5.15.0-1106-kvm-x86_64-with-glibc2.35, 1 CPU(s), Python 3.10.12 (64-bit), pyperformance 1.14.0.

1. Overview

Purpose

nbody comes from the Computer Language Benchmarks Game. It simulates the orbits of the Jovian planets (Sun, Jupiter, Saturn, Uranus, Neptune) under Newtonian gravity. It is a classic "pure floating-point number crunching" workload: no I/O, no allocation of user objects, no strings -- only arithmetic on Python floats stored in Python lists. It therefore measures how fast the CPython interpreter executes numeric bytecode.

Libraries

- Python standard library only. No numpy / C extensions.
- pyperf: timing harness only (perf_counter, worker processes, statistics); it is not part of the measured computation.

Data structures

- BODIES : dict name -> (position, velocity, mass)
 - position = [x, y, z] (mutable list of 3 floats)
 - velocity = [vx, vy, vz] (mutable list of 3 floats)
 - mass = float (in units where $G = 1$, $SOLAR_MASS = 4\pi^2$)
- SYSTEM : list of the 5 body tuples (the lists inside are shared by reference, so in-place updates are visible everywhere).
- PAIRS : list of the 10 unordered body pairs, built once by a pure-Python re-implementation of `itertools.combinations(SYSTEM, 2)`.

Algorithm

Direct all-pairs $O(N^2)$ gravity with a semi-implicit (symplectic) Euler integrator, $dt = 0.01$:

```
for each step:
    for each pair (i, j):
        d = r_i - r_j
        mag = dt * |d|^-3
        v_i -= d * m_j * mag
        v_j += d * m_i * mag
    for each body:
        r += dt * v                (uses the NEW velocity -> symplectic)
```

`report_energy()` computes total energy (kinetic + potential) before and after the run as a sanity check; `offset_momentum()` makes the total momentum zero by adjusting the Sun's velocity.

One benchmark "loop" = `report_energy` + 20,000 steps + `report_energy`.
 Per step: 10 pair interactions + 5 position updates, i.e. 200,000 pair interactions per loop. `advance()` is >99.9% of the work: `report_energy()` evaluates the 10 pairs once, `advance()` evaluates them 20,000 times.

Why this is slow in CPython

A float in CPython is a heap object (`PyFloatObject`, 24 bytes). Every arithmetic bytecode goes: `dispatch` in `_PyEval_EvalFrameDefault` -> `PyNumber_Multiply` -> `float_mul` -> `PyFloat_FromDouble` (free-list allocation) -> `Py_DECREF` of temporaries. The actual double multiply is ~1 machine instruction; the surrounding interpreter work is tens to hundreds. In addition, every `v[i] -= x` needs `BINARY_SUBSCR` + `INPLACE_SUBTRACT` + `STORE_SUBSCR` with stack shuffling (`DUP_TOP_TWO`, `ROT_THREE`), and every pair iteration needs `FOR_ITER` and three `UNPACK_SEQUENCE` operations.

2. Initial analysis (profiling)

Method (all automated in script_nbody.sh)

- `pyperformance run -b nbody` -> `results/baseline.json`
- `perf stat -r 5 (instructions, branches, cache)` -> `results/perf_stat_original.txt`
- `perf record -e cpu-clock -F 999 -g -- python3-dbg -m pyperformance run --bench nbody (course guide method)` -> `results/perf_report_original*.txt`
- `FlameGraph (stackcollapse-perf.pl | flamegraph.pl)` -> `flamegraphs/flamegraph_nbody_original.svg`
- `bytecode_count_nbody.py (executed opcodes/step)` -> table below

`python3-dbg` is used so `perf` can resolve CPython's internal C symbols. Python 3.10 has no `-X perf` support, so the `perf` flame graph shows interpreter C functions, not Python function names.

VM note: the QEMU vCPU exposes no "cycles" event, so `perf`'s default sampling event records zero samples; we sample on `cpu-clock` instead. `perf stat` runs `profile_nbody.py` (`bench_nbody()` called directly in one process) so counters exclude `pyperf` worker spawning and `venv` setup.

Baseline result

`pyperformance run -b nbody`: 229 ms +- 3.0 ms per loop (60 values)
 (under `python3-dbg` while profiling: 481 ms +- 3.2 ms)

```
perf report -- hottest symbols, self overhead (original)
overhead  symbol
22.29%    _PyEval_EvalFrameDefault
2.20%     binary_op1
2.03%     validate_list
1.88%     _memset_avx2_unaligned_erms
1.86%     _PyMem_DebugCheckAddress
1.61%     PyFloat_FromDouble
1.47%     float_mul.lto_priv.0
1.46%     lookdict_unicode_nodummy
1.37%     _PyPegen_is_memoized
1.32%     float_dealloc.lto_priv.0
1.15%     read_size_t
1.05%     _PyObject_Malloc
0.98%     list_ass_item.lto_priv.0
0.98%     _PyUnicode_CheckConsistency
0.96%     PyNumber_AsSsize_t

grouped by role:
22.29%    bytecode dispatch (_PyEval_EvalFrameDefault)
7.14%     list/tuple indexing (v[0], unpacking)
6.95%     float objects: arithmetic + alloc/free
0.64%     pow() in libm
4.27%     number-protocol dispatch (binary_op1 ...)
4.75%     object alloc / refcount bookkeeping
9.98%     debug-build self-checks (python3-dbg only)
2.56%     dict lookups (globals/attributes)
(98.1% of all samples are in the benchmark worker process; the rest
is pyperformance start-up. Shares below 100% in total are the long tail
of smaller symbols.)
```

```
perf stat (original), per run of 5 loops x 20,000 steps
instructions      8,713,724,770
branches          1,500,031,830
branch-misses      6,778,605    0.45% of branches
cache-references    5,427,800
cache-misses       159,132    2.93% of cache refs
task-clock         1266.7 ms
cycles             not exposed by the VM
```

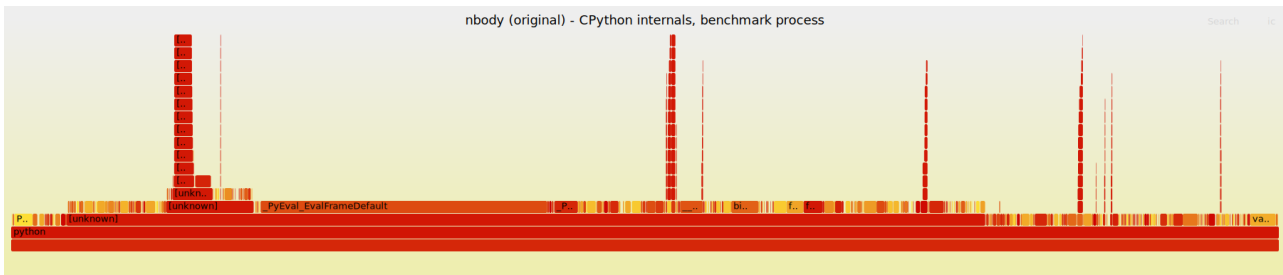
Interpretation: cache misses are negligible (159,132 for 8.7 billion instructions; the whole simulation state is 5 bodies, a few KB) and branch misses are negligible (0.45%). The workload is interpreter-overhead bound: ~87,137 machine instructions per simulation step, i.e. ~65 x86 instructions per executed bytecode -- for arithmetic whose real work (a double add/mul) is ~1 instruction. Speedup must come from executing fewer bytecodes, not from memory layout.

Executed bytecode per simulation step (original)

```
LOAD_FAST          397.0
STORE_FAST         186.0
BINARY_MULTIPLY    135.0
LOAD_CONST         85.0
DUP_TOP_TWO        75.0
BINARY_SUBSCR      75.0
ROT_THREE          75.0
STORE_SUBSCR       75.0
UNPACK_SEQUENCE    60.0
INPLACE_ADD        45.0
BINARY_SUBTRACT    30.0
INPLACE_SUBTRACT   30.0
BINARY_ADD         20.0
FOR_ITER           18.0
JUMP_ABSOLUTE      16.0
BINARY_POWER       10.0
GET_ITER           2.0
TOTAL              1334.1
```

Bottlenecks identified

- B1 Container access: 150 list subscripts + 150 stack-shuffle opcodes per step exist only because body state lives inside lists.
- B2 Loop/unpacking overhead: FOR_ITER + 60 UNPACK_SEQUENCE per step to walk PAIRS and destructure ((pos, vel, m), (pos, vel, m)).
- B3 Redundant arithmetic: `dt \* ...` is recomputed for every pair although dt and masses are constant during the run.
- B4 Boxed-float arithmetic itself (~270 arithmetic opcodes per step, each allocating a new PyFloat). This is the irreducible cost in pure Python and the target of the hardware proposal (section 5).



Flame graph - nbody, original (perf record -e cpu-clock -g, python3-dbg). Interactive version: [benchmarks/nbody/flamegraphs/flamegraph_nbody_original.svg](#)

3. Optimizations

All changes are in `benchmarks/nbody/optimized/run_benchmark.py`. The public API (advance, report_energy, offset_momentum, bench_nbody), the workload (5 bodies, 20,000 steps, $dt = 0.01$) and the benchmark name are unchanged, so it runs under pyperformance through a custom manifest (`benchmarks/nbody/manifest/MANIFEST`) and compares directly with the original.

- 01 Precompute $m*dt$ (targets B3)
 $mdt_k = m_k * dt$ is computed once per call. The pair loop then uses $mag = |d|^{\sim 1.5}$ and $b1m = mdt_i * mag \rightarrow$ one fewer multiply per pair.
- 02 Body state in local variables (targets B1)
 At the start of `advance()`, positions/velocities are unpacked into 30 local variables (`x0..z4, vx0..vz4`); they are written back to the lists once at the end. Locals live in the frame's `fastlocals` array, so access is `LOAD_FAST/STORE_FAST` (an array index) instead of list subscripting.
- 03 Specialization / loop unrolling (targets B2)
 Since the number of bodies is fixed during a run, the 10 pair interactions are generated as straight-line code. The generator runs once at import time for `len(SYSTEM)` bodies (so the code is general for any N) and is compiled with `compile()/exec()`; nothing is generated in the timed region. `'python3 run_benchmark.py --dump-generated'` prints it. This is the same idea a JIT uses (specializing code for values that are constant at run time). 02 is only possible together with 03: without unrolling there is no way to name "the velocity of body j " as a local variable.

Correctness (verify_nbody.py, 20,000 steps from identical initial state)
`steps=20000`
 energy before : original=-0.1690751638285245 optimized=-0.1690751638285245
 energy after : original=-0.16908926275527172 optimized=-0.16908926275527195
 $|dE|=2.220e-16$ max $|state\ diff|=6.117e-12$
 PASS
 A $\sim 1e-16$ energy difference is expected: 01 computes $(m*dt)*mag$ instead of $m*(dt*mag)$, and IEEE-754 multiplication is not associative.

Effect on executed bytecode (per step)

opcode	original	optimized
LOAD_FAST	397.0	385.8
STORE_FAST	186.0	136.6
BINARY_MULTIPLY	135.0	125.0
LOAD_CONST	85.0	10.4
INPLACE_ADD	45.0	45.0
STORE_SUBSCR	75.0	0.3
BINARY_SUBSCR	75.0	0.1
ROT_THREE	75.0	0.0
DUP_TOP_TWO	75.0	0.0
UNPACK_SEQUENCE	60.0	0.2
INPLACE_SUBTRACT	30.0	30.0
BINARY_SUBTRACT	30.0	30.0
BINARY_ADD	20.0	20.0
BINARY_POWER	10.0	10.0
FOR_ITER	18.0	0.0
JUMP_ABSOLUTE	16.0	1.0
GET_ITER	2.0	0.0
EXTENDED_ARG	0.0	1.0
TOTAL	1334.1	795.5 (-40%)

Container-access, stack-shuffle and unpacking opcodes disappear; the float arithmetic opcodes remain (minus the dt multiplies removed by 01).

Rejected / negative results (kept in `ablation_nbody.py` on purpose)

- R1 Replace $x^{\sim -1.5}$ with $1/(d2*\sqrt{d2})$. Common advice in C, but in CPython it saves a `libm pow()` call while adding a `CALL_FUNCTION` to `math.sqrt` plus an extra division opcode -- interpreter overhead cancels the win (see ablation table).
- R2 numpy vectorization. With only 5 bodies / 10 pairs per step, each numpy call's fixed overhead (argument parsing, array allocation, dtype dispatch) dominates the tiny amount of actual arithmetic; numpy only wins when N is large. This is the same "offload granularity" trade-off that applies to the hardware accelerator in section 5.

4. Performance comparison

```
pyperformance (results/compare.txt)
original : 229 ms +- 3.0 ms
optimized: 140 ms +- 4.9 ms
-> 1.64x faster, run time -39% (meets the >= 7% requirement)
Same trend under python3-dbg while profiling: 481 ms -> 274 ms (1.76x).
```

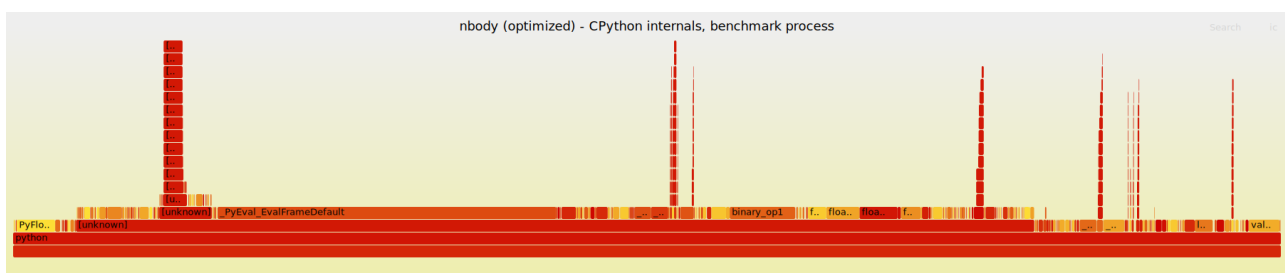
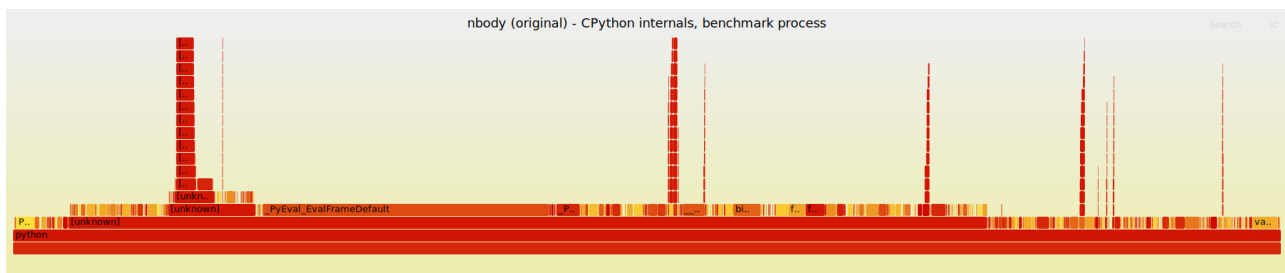
```
Ablation (ablation_nbody.py) -- each optimization measured separately
variant                time / 20k steps  vs original
original               228.7 +- 3 ms      1.00x
R1 sqrt instead of pow 229.4 +- 3 ms      1.00x
O1 precompute m*dt     226.0 +- 4 ms      1.01x
O1+O2+O3 (final)      139.3 +- 4 ms      1.64x
(R2 numpy skipped: numpy is not installed in this environment)
```

```
perf stat before/after (5 loops x 20,000 steps per run, mean of 5 runs)
              original      optimized  change
instructions  8,713,724,770  5,568,251,540  -36.1%
branches      1,500,031,830  961,259,560   -35.9%
branch-misses  6,778,605        5,315,149     -21.6%
cache-misses   159,132        24,773        -84.4%
task-clock     1266.7 ms      789.5 ms      1.60x faster
elapsed        1.657 s        1.108 s        1.50x faster
```

Instructions drop by 36% (~87,137 -> ~55,683 per step), in line with the 40% drop in executed bytecodes (1334 -> 795 per step). The time ratio (1.60x) tracks the instruction ratio (1.56x): the gain is "less interpreter work", not better cache or branch behaviour.

```
perf profile before/after (flame graphs: flamegraphs/flamegraph_nbody_*.svg)
Estimated absolute cost = self% x total sampled time of each profile.
role                original  optimized  change
bytecode dispatch (_PyEval_EvalFrameDefault)  22.3%    26.1%    -49%
list/tuple indexing (v[0], unpacking)         7.1%     1.1%    -93%
float objects: arithmetic + alloc/free        7.0%    13.5%    -15%
pow() in libm                                  0.6%     1.6%     +8%
number-protocol dispatch (binary_op1 ...)     4.3%    10.2%     +4%
object alloc / refcount bookkeeping           4.7%     5.1%    -54%
debug-build self-checks (python3-dbg only)    10.0%     8.4%    -64%
dict lookups (globals/attributes)             2.6%     2.8%    -52%
```

Largest absolute reductions: `_PyEval_EvalFrameDefault` (-49%), `__memset_avx2_unaligned_erms` (-76%), `_PyPegen_is_memoized` (-100%), `_PyMem_DebugCheckAddress` (-72%), `list_ass_item.lto_priv.0` (-100%).
Practically gone from the optimized profile: `list_ass_item.lto_priv.0`, `_PyPegen_is_memoized`, `PyNumber_AsSsize_t`.
Top of the optimized profile: `_PyEval_EvalFrameDefault` (26.1%), `binary_op1` (5.2%), `PyFloat_FromDouble` (3.4%), `float.mul.lto_priv.0` (2.9%), `float.dealloc.lto_priv.0` (2.5%).
Total sampled time: original 42.18 s, optimized 18.30 s (cpu-clock event count, includes pyperformance start-up).



5. Hardware acceleration proposal

One accelerator serves both benchmarks: V3A, a 3D Vector-math Accelerator. The hot arithmetic of nbody (pair distance, $|d|^3$) and of raytrace (ray-sphere test: dot products and a square root) is the same small set of operations: 3D subtract, dot product and inverse square root. V3A implements them once as reusable pipelined IP and exposes two kernels on one PCIe card. Full RTL: hw/rtl/*.v (Verilog-2005/SystemVerilog-compatible, Icarus-tested).

5.1 Hardware description (hw/rtl)

```
v3a_top.v           PCIe core logic: register bus decode, IRQ
v3a_nbody_ctrl.v    nbody kernel: body registers, step FSM, integrator
v3a_pair_pipe.v      pairwise gravity datapath, 19 stages
v3a_ray_ctrl.v       ray kernel: ray BRAM, feed FSM, reducer, results
v3a_raysphere_pipe.v ray-sphere datapath, 19 stages
v3a_invsqrt.v        shared inverse-square-root IP, 12 stages
v3a_delay.v          sideband pipeline balancing
v3a_common.vh        Q23.40 fixed-point multiply
v3a_invsqrt_lut.vh   64-entry initial-guess ROM (generated)
Golden model: hw/model/v3a_model.py (bit-accurate). Testbench:
hw/tb/tb_v3a.v. Everything is run by: python3 hw/run_hw_tests.py
```

5.2 Inputs and outputs

```
Number format : signed Q23.40, 64 bits (range +/-8.4e6, LSB 9.1e-13).
                  Chosen instead of IEEE-754 double: a fixed-point multiply
                  needs no normalization/rounding logic (about half the area
                  of a double-precision FPU multiplier, one stage per op).

Clock           : single clock, target 100 MHz (see 5.7), sync. reset.
Host bus        : memory-mapped, word-addressed, 64-bit data (PCIe BAR0
                  behind an AXI4-Lite bridge; BAR byte offset = 8 x index).
                  24-bit address: 0x000000 nbody window, 0x100000 ray window.

Interrupt       : irq (level) when a job is done -> MSI.
nbody kernel    : in  : NBODIES (2..8), NSTEPS, DT, per body x,y,z,vx,vy,vz,m*dt
                  out: updated x,y,z,vx,vy,vz after NSTEPS steps; STEPDONE,
                  CYCLES (performance counter)
ray kernel      : in  : NRAYS (1..4096), NSPH (1..16), EPS, MODE
                  (0 closest hit, 1 any hit), spheres c,r^2,
                  rays p,d (d normalized)
                  out: per ray (sphere index or -1, t) / (occluded 1/0)
Pipeline IP     : valid/data streaming, no back-pressure needed (the
                  controllers always accept every output).
v3a_invsqrt     in  x (Q23.40)                out y = x^-1/2
v3a_pair_pipe   in  r_i, r_j, m_i*dt, m_j*dt, i, j out f_i, f_j, i, j
v3a_raysphere_pipe in  p, d, c, r^2, ray id, sph id out disc, t, ids
```

5.3 Hardware architecture

Datapath (see hw/docs/v3a_pipelines.png):

- Every stage has at most ONE 64x64 multiply followed by a register, so the critical path is the same everywhere and throughput is one result per clock cycle after the 19-cycle latency.
- v3a_invsqrt: stage 1 normalizes $x = m^4 k$ with m in $[1,4)$ using a priority encoder and a barrel shift; stage 2 takes y_0 from a 64-entry ROM indexed by the top 6 bits of m ; stages 3-11 run three Newton iterations $y \leftarrow y(3 - m y^2)/2$ (3 stages each: $y^2, m y^2$, update); stage 12 shifts back by k . Measured accuracy: relative error $< 2.3e-7$ over $1e-6..1e6$ (limited by the Q23.40 LSB), $\sim 1e-12$ for typical values.
- Sideband data (indices, d , masses, v) travel through v3a_delay shift registers so they stay aligned with the invsqrt result.

Control logic:

- v3a_nbody_ctrl FSM per time step: CLR (zero accumulators) -> FEED (one pair (i,j) per cycle into the pipeline) -> WAIT (outputs accumulate $acc_i \leftarrow f_i, acc_j \leftarrow f_j$) -> VEL ($v \leftarrow acc$ for all bodies) -> POS ($r \leftarrow dt*v$, one body per cycle) -> NEXT (repeat NSTEPS times -> IRQ). All pairs of a step use the positions from the start of the step, so pairs are independent and can be streamed through the pipeline.
- v3a_ray_ctrl: a feed FSM walks (ray r , sphere s) and streams one test per cycle (ray words come from a synchronous BRAM, 1-cycle read); a reducer after the pipeline keeps the running best hit of the current ray ($t > -EPS$ and smallest t ; or any $t > EPS$ for shadow rays) and writes the result BRAM when the last sphere of that ray comes out.

Verification (hw/run_hw_tests.py, vectors from the real benchmark code):

```
nbody : 5 bodies, 2000 steps, 10 pairs/step
RTL vs golden fixed-point model: BIT-EXACT
energy: float benchmark -0.169071606870, hardware -0.169071606646
        relative difference 1.33e-09
max |position diff| vs float benchmark: 9.01e-07
cycles: 76000 = 38.0 per step (10 pair feeds +
        19 latency + 5 position updates + control)
ray    : closest hit, primary rays
        2500 rays x 7 spheres = 17500 tests
RTL vs golden fixed-point model: BIT-EXACT
```

```

closest sphere == float benchmark: 2500/2500 rays
(194 hits), max |t error| 1.12e-09
cycles: 17521 = 1.001 per test (+19 latency, +1 BRAM read)
ray      : any hit, shadow rays
1940 rays x 7 spheres = 13580 tests
RTL vs golden fixed-point model: BIT-EXACT
shadow decision == float benchmark: 1940/1940 rays
(53 occluded)
cycles: 13601 = 1.002 per test (+19 latency, +1 BRAM read)
RESULT: PASS

```

5.4 Hardware/software interface

```

Driver      : Linux PCIe driver, character device /dev/v3a. mmap() of BAR0
              for the registers; pinned DMA buffers for bulk data; MSI
              interrupt wakes the waiting process.
Transfers   : small state (nbody: 5 bodies x 7 words = 280 bytes) is written
              directly through BAR0. Ray batches (48 bytes per ray, up to
              4096 rays) are copied by the card's AXI4 DMA master from a
              pinned host buffer into the ray BRAM; results are copied back
              the same way (the RTL models the BRAM filled via the bus).
API         : Python extension module (C, or ctypes over the ioctl):
              v3a.nbody_advance(bodies, dt, steps)
              v3a.closest_hits(rays, spheres, eps) -> [(index, t), ...]
              v3a.any_hits(rays, spheres, eps)    -> [occluded, ...]
              It converts floats <-> Q23.40, fills buffers, writes CTRL,
              sleeps until the IRQ and reads the results.
Key rule    : offload granularity. One PCIe round trip costs tens of
              microseconds, while one CPython pair/intersection costs
              ~0.4-0.7 microseconds. Calling the card per pair or per ray
              would be 10-100x SLOWER than software. Every job is therefore
              a batch: all NSTEPS of nbody in one call; all rays of one
              recursion level ("wavefront") in one call for raytrace.
              (Same lesson as numpy in section 3: fixed per-call overhead
              must be amortized over many elements.)

```

5.5 Acceleration justification (nbody)

Why a good candidate: `advance()` is >99.9% of the benchmark run time and is a small, regular, data-independent loop of float arithmetic (no branches, no allocation, tiny state). After the software optimization the remaining cost is boxed-float arithmetic, which no Python change can remove, but which maps 1:1 onto a fixed-point datapath.

Software changes: `advance(dt, n)` calls `v3a.nbody_advance()` once with the whole state; `report_energy()` and `offset_momentum()` stay in Python.

Estimate (assumptions: 100 MHz, measured 38 cycles per step for 5 bodies in RTL simulation, PCIe setup + 2 x 280-byte transfers + IRQ < 0.1 ms):

```

hardware time per loop = 20,000 steps x 38 cycles x 10 ns = 7.6 ms
CPython original      : ~229 ms / loop -> ~30x faster
CPython optimized     : ~140 ms / loop -> ~18x faster

```

(CPython numbers from results/baseline.json and optimized.json on the course VM; the whole benchmark loop speeds up by almost the same factor, because `advance()` dominates.)

Scaling: cycles per step = $N(N-1)/2 + N + \sim 23$. With $N = 5$ the 19-cycle latency dominates; for $N = 100$ bodies: 5,073 cycles = 51 μ s per step vs ~3.5 ms for optimized CPython (~68x). Replicating the pair pipeline P times divides the pair part by P (pairs within a step are independent).

Honest limit: against compiled C on a modern core (~20-50 ns per pair) a single pipeline at 100 MHz (10 ns per pair) is only a few times faster; the large factors above are relative to the CPython interpreter.

Accuracy: RTL results are bit-exact to the golden model; the fixed-point trajectory stays within $\sim 1e-6$ of the double-precision benchmark after 2,000 steps and the total energy differs by $\sim 1e-9$ (relative).

5.6 Block diagram

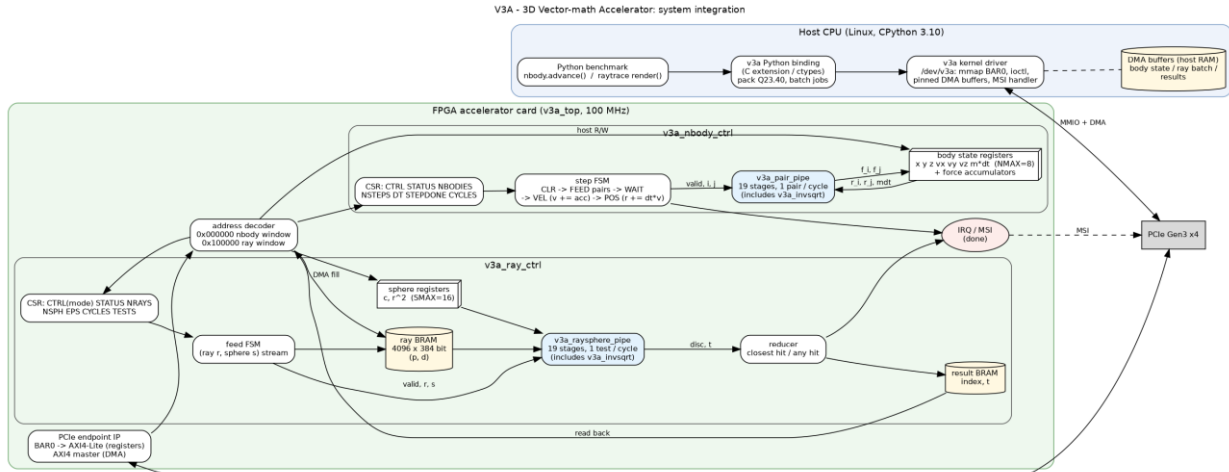


Figure 1 - V3A accelerator: system integration (host, driver, PCIe, both kernels)

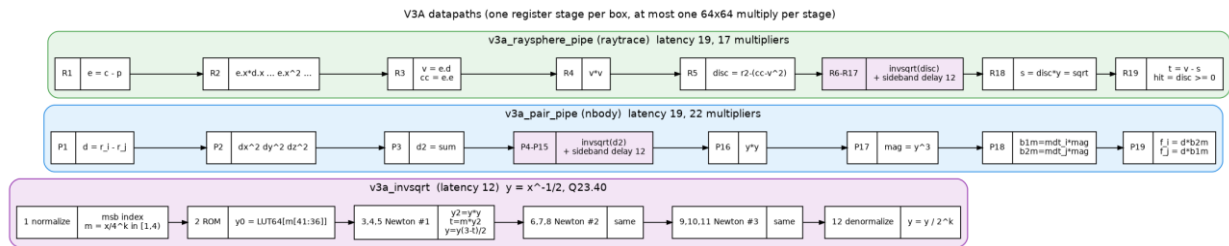


Figure 2 - V3A datapaths: nbody pair kernel, ray-sphere kernel, shared inverse-sqrt IP

hw/docs/v3a_system.png system: host, driver, PCIe, both kernels
 hw/docs/v3a_pipelines.png datapaths (pair pipeline and invsqrt stages)

```

Python advance() --> v3a ext --> driver --MMIO/DMA--> PCIe EP --> decoder
      ^
      |
      +----- updated state <--- body regs <--- nbody FSM --> pair pipe
      |               (x,v,m*dt)          ^   |   (19 st.) |
      |               ^                   |   v           |
      +--- acc += f_i, f_j <-----+
  
```

5.7 Performance / area / power trade-offs

Frequency: one 64x64 multiply per stage. On an FPGA (e.g. UltraScale+) each multiply spans ~9 DSP blocks and needs ~3 internal pipeline registers to close ~150-200 MHz; that adds latency (19 -> ~50 cycles) but keeps 1 result/cycle. We assume a conservative 100 MHz; a 28 nm ASIC could run ~250 MHz.

Area (order of magnitude): nbody kernel 25 multipliers (22 in the pipeline + 3 in the integrator), ray kernel 17 -> ~42 64x64 multipliers, ~400 DSP blocks, ~30k flip-flops, ~44 BRAM36 for the ray buffer; in 28 nm roughly 1.5M gates (~2-3 mm²).

Alternatives considered:

- Iterative datapath (one shared multiplier + microcode): ~20x less area, but ~25 cycles per operation -> ~1000 cycles per nbody step, i.e. about the SAME speed as CPython. Rejected: area saved, benefit lost.
- Fully parallel/systemic array (one pair unit per pair): 10x the area for 5 bodies, removes the 10-cycle feed time only; the 19-cycle latency still dominates. Not worth it for N = 5, attractive for large N.
- IEEE-754 double instead of Q23.40: bit-exact with Python, but ~2x area per multiplier and deeper pipelines; the measured fixed-point error (~1e-9 relative energy) is far below what matters for the simulation.
- Sharing one invsqrt between both kernels: saves 9 multipliers, but the kernels could no longer run at the same time.

Power: dominated by the DSP/multiplier toggling while busy (a few watts on an FPGA card at 100 MHz); clock gating when idle (STATUS.busy = 0) makes average power proportional to use. Energy per nbody loop: ~7.6 ms x a few W ~ tens of mJ, vs ~139 ms x tens of W for a CPU core running CPython.

5.8 Known simplifications of this RTL (deliberate, and what a real card adds)

- Clock domains: the design uses one clock. A real PCIe card runs the user logic on its own clock and needs asynchronous FIFOs between the PCIe core clock and the kernel clock; that CDC logic is not modelled here.
- DMA: the RTL fills the ray BRAM through the register bus (the testbench acts as the driver). A real card uses an AXI4 master (XDMA/QDMA IP) to read the pinned host buffer; the register map and the FSMs do not change.
- Ray batch size: the ray BRAM holds 4096 rays, while a 100x100 frame has

- 10,000 primary rays, so the driver issues ~3 batches per wavefront level. This is already included in the per-batch overhead of the estimate.
- Fixed-point overflow is not detected: Q23.40 wraps silently above $+8.4e6$. Values in both benchmarks stay far below that (the largest intermediate is $|d|^2 < 3e3$), but a production version would either saturate or raise a status flag, at the cost of extra logic per stage.
 - nbody body count is limited to $NMAX = 8$ (state lives in registers so every pair read is single-cycle). More bodies would move the state into BRAM and add a load stage to the pipeline.
 - No synthesis or timing closure was run (not required): the frequency numbers are estimates based on one 64x64 multiply per pipeline stage.

6. Conclusion

The nbody benchmark is interpreter-overhead bound, not memory- or branch-bound. Restructuring the hot loop so that body state lives in local variables and the fixed set of pair interactions is unrolled removes 40% of the executed bytecodes and gives a 1.64x speedup under pyperformance with identical physics. What remains is boxed float arithmetic, which no pure-Python change can remove; this is exactly the part targeted by the hardware accelerator in section 5.